

BPFix: Rust-Style Diagnostics for eBPF Verifier Failures

Anonymous Author(s)

Abstract

eBPF programs must pass a complex static verifier whose rejection messages are notoriously opaque—developers receive verbose, thousand-line logs pinpointing symptoms but obscuring the underlying cause. Fixing these rejections requires understanding *where* the safety proof broke, not just *that* it broke: the source code may contain the intended safety argument while the verifier’s abstract domain loses track of it through LLVM’s lowering.

We observe that the verifier’s execution trace is a complete record of a proof attempt—a sequence of per-instruction abstract states. By analyzing *abstract state transitions* across this trace (bounds collapses, type downgrades, provenance loss), BPFix automatically locates where the safety proof broke and why, without requiring knowledge of which specific safety condition was checked. The transition pattern itself classifies the failure: a safety property never established indicates a source bug; one established then lost indicates a compiler-induced lowering artifact.

BPFix operates entirely in userspace and requires no kernel modifications.

CCS Concepts: • Software and its engineering → Compilers; Software verification and validation; • Computer systems organization → Operating systems.

Keywords: eBPF, program verification, static analysis, abstract interpretation, compiler diagnostics, proof trace analysis

ACM Reference Format:

Anonymous Author(s). 2027. BPFix: Rust-Style Diagnostics for eBPF Verifier Failures. In *Proceedings of ACM European Conference on Computer Systems (EuroSys ’27)*. ACM, New York, NY, USA, 7 pages.

1 Introduction

eBPF is the foundation for critical kernel extensions powering security enforcement (Cilium, Katran), performance tracing (bcc, bpftrace), and networking infrastructure in production Linux deployments. Ensuring safety, however, requires each eBPF program to pass a mandatory static verifier, an abstract interpreter that rigorously checks all instructions before execution. Unfortunately, when verification fails, developers confront overwhelmingly verbose diagnostic logs often spanning thousands of lines. These logs reveal only symptoms at rejection points, leaving the root cause—typically a safety proof destroyed deep within compiler-generated

instructions—buried hundreds of lines earlier. Consequently, debugging verifier rejections has become the single largest productivity bottleneck in eBPF development, as confirmed by a recent Linux Foundation security audit [10] and extensive developer frustration on Stack Overflow [2].

Verifier-related bug reports and workaround commits repeatedly show proof-reshaping workarounds: the source code may be correct, but the verifier’s abstract domain cannot track the developer’s safety argument through LLVM’s lowering. These rejections are inherent to the verifier’s design and cannot be eliminated by better diagnostics alone—but fixing them correctly requires knowing *where* the safety proof broke, which current tools cannot provide. We find that existing solutions fundamentally misunderstand the structure embedded within verification traces: current tools either rely on shallow regular expressions applied only to the final rejection message [9] or feed entire raw logs to general-purpose language models [16], neither approach recognizing the verifier trace as a structured safety proof attempt.

Our key insight is that the verifier’s execution trace is a complete record of a proof attempt—a sequence of per-instruction abstract states—and that analyzing *abstract state transitions* across this trace reveals where and why the safety proof broke. The error message identifies which safety condition was checked (we call this the required proof, used as a focus mechanism), but the core analysis is detecting the instruction whose abstract state transition—bounds collapse, type downgrade, provenance loss—caused the violation. We call this the “proof-loss point.” Crucially, the transition pattern itself classifies the failure: a safety property never established in the trace indicates a source-level programming bug, while one established then lost indicates a compiler-induced lowering artifact.

Motivated by this insight, we design and implement BPFix, a userspace diagnostic engine that detects safety-relevant abstract state transitions across verifier traces, traces their causal chains back to the root-cause instruction, and provides concise, Rust-inspired diagnostics pinpointing exactly where and why the safety proof broke. Section 4 defines the methodology used to evaluate reliability, proof reconstruction, localization, object correlation, baseline comparison, and latency.

In summary, this paper makes three main contributions:

- Identifies a structured, generalizable framework (abstract state transition analysis) that precisely pinpoints verifier root-cause failures and systematically classifies failure types (§3.3–§3.5).

Panel A: Source code**Panel B: Raw verifier output (excerpt)****Panel C: BPFix output**

```

111 1 int parse_ext(struct xdp_md *ctx) {
112 2     void *data = (void*)(long)ctx->data;
113 3     void *data_end = (void*)(long)ctx->
114   data_end;
115 4     if (data+sizeof(struct ext) > data_end)
116 5         return XDP_DROP; // bounds
117 6     struct ext *e = data;
118 7     // reads guarded by line 3
119 8     __u16 len = __bpf_htons(e->len); // byte-
120 9     data += len; // REJECTED
121 10    /* "unbounded min value" */
122 11 }

```

```

22: R0_w=inv(id=0,umax=65280,
var_off=(0x0; 0xffff00))
R6_w=inv(id=0,umax=255,
var_off=(0x0; 0xff))
22: (4f) r0 |= r6
23: R0_w=inv(id=0) <-- bounds GONE
...
math between pkt pointer and register
with unbounded min value is not allowed

```

Pretty Verifier output:

"The value added to the packet pointer has an unbounded minimum. Consider adding `& 0xFFFF` to bound the value."

```

error[E005]: lowering_artifact
-- parse_ext.c
|
3 | if (data+sizeof(ext) > data_end)
| ~~~~~
| proof established
| R5: pkt(off=0) -> pkt(off=4,r=4)
|
7 | __u16 len = __bpf_htons(e->len);
| ~~~~~
| proof lost: OR destroys bounds
| R0: scalar(umax=65280) -> scalar
|
8 | data += len;
| ~~~~~
| rejected: pkt_ptr + unbounded
|
= note: LLVM lowers htons() to byte-load,
shift, OR. Verifier cannot track OR result.
= help: Add clamp: len &= 0xFFFF;

```

Figure 1. The motivating example: SO #70750259. **Panel A:** source code with a correct bounds check at line 3. **Panel B:** the verifier’s raw output (60+ lines, excerpt shown) and Pretty Verifier’s single-line response—both pointing at the symptom, not the cause. **Panel C:** BPFix’s Rust-style multi-span output tracing abstract state transitions through three source locations: proof established (line 3), proof lost (line 7, OR instruction destroys bounds), rejected (line 8).

- Designs and implements BPFix, a fast userspace diagnostic tool leveraging this framework to deliver concise, actionable verifier diagnostics (§3.2–3.5).
- Defines an evaluation methodology for measuring diagnostic reliability, required-proof reconstruction, localization quality, object-CFG correlation, baseline comparisons, and runtime overhead (§4).

2 Background

2.1 The eBPF Verification Pipeline

eBPF programs are written in C or Rust, compiled by LLVM to BPF bytecode, and submitted to the kernel via the `bpf()` syscall. The Linux verifier runs before any program executes. It performs a dataflow-style abstract interpretation [1, 4]: it walks all execution paths through the bytecode, maintaining per-register abstract state and enforcing a set of safety properties at each instruction. Programs that violate any property are rejected.

The key architectural point is that there are three distinct reasoning levels:

- **Source level.** The programmer reasons about C/Rust values, bounds checks, null checks, and pointer lifetimes.
- **Compiler level.** LLVM transforms source into BPF bytecode, potentially splitting, merging, or reordering computations.
- **Verifier level.** The verifier reasons about BPF register abstract state: types, scalar bounds (umin/umax, smin/smax), pointer offsets, packet ranges, and provenance IDs.

A proof that is valid at the source level can be lost at the verifier level if LLVM’s lowering produces register state that the verifier’s abstract domain cannot represent. This is the root cause of lowering artifacts.

2.2 Anatomy of a LOG_LEVEL2 Trace

When `log_level=2` is set in `BPF_PROG_LOAD`, the verifier emits a complete per-instruction trace interleaving per-register abstract state (type, scalar bounds, pointer offset, packet range, provenance ID), BTF source annotations [8], `mark_precise` backtracking chains, and control flow merge points. A typical 50-instruction program produces 100–300 lines. We detail the parsing of each element in §3.2.

3 Design: The BPFix Diagnostic Engine

3.1 Architecture Overview

BPFix is a five-stage pipeline that transforms a raw LOG_LEVEL2 verifier trace into a structured multi-span diagnostic. Figure 2 shows the architecture.

Each stage is independently testable and produces a well-defined intermediate representation.

3.2 State Trace Parsing

The log parser (Stage 1) handles error message identification: it matches the final error line against error patterns and maps each pattern to a taxonomy class (*source_bug*, *lowering_artifact*, *verifier_limit*, *env_mismatch*, *verifier_bug*).

The state trace parser (Stage 2) parses the remaining structure. For each BPF instruction, it extracts:

- Instruction index, opcode, and mnemonic

Verifier LOG_LEVEL2 trace

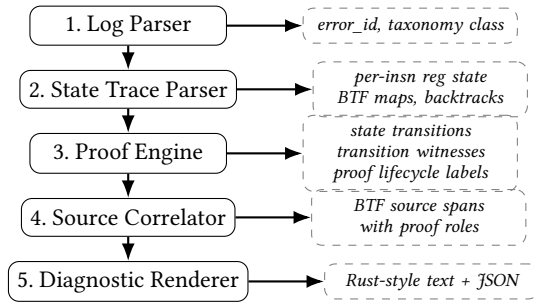


Figure 2. BPFix five-stage pipeline. bpf_{fix} owns the diagnostic product layer, while bpf_{analysis} provides verifier-log and BPF bytecode analysis facts.

- Pre/post-state per live register (type, umin/umax, smin/smax, offset, packet range, provenance ID)
- BTF source annotation (file, line, column) if present
- Backtracking links from mark_{precise} annotations
- Branch merge points with state at each join

The primary challenge is format variability. Register state can appear before or after the instruction, in branch merge lines, in backtracking annotations, or across multiple lines. The parser normalizes all variants into a uniform TracedInstruction representation.

Backtracking chain extraction. The verifier’s own mark_{precise} backward pass is emitted in the log as `regs=R stack=S before K: (op) ... annotations`. BPFix extracts these into a BacktrackChain structure: a sequence of (instruction_index, registers_tracked) pairs. This is the verifier’s own root-cause chain, expressed as debug text.

3.3 Required-Proof Reconstruction

The verifier trace contains thousands of abstract state transitions per execution path; analyzing all of them would produce noise. The required proof serves as a *focus mechanism*: given the error message and the register state at the rejection point, BPFix infers which safety condition the verifier needed—the *required proof*—expressed as a formal predicate over register state. This predicate then directs the state-transition analysis in Section 3.4 to the relevant registers and state dimensions.

BPFix supports the verifier proof families shown in Table 1. The error message identifies the proof *family*. The register state at the rejection point provides concrete parameters. BPFix instantiates the predicate template with these parameters.

This predicate instantiation is not pattern matching on error text. The same error message (invalid access to packet) with different register states at the rejection point produces different predicate instances with different parameters. The predicate is grounded in the verifier’s type system: for packet access, the verifier literally checks

Table 1. Verifier proof families reconstructed by BPFix.

Family	Predicate	Example error
packet_access	type==pkt && off+sz<=r	invalid access to packet
map_value_bounds	0<=off && off+sz<=val_sz	invalid map value access
null_check	type!=*_or_null	map_value_or_null deref
stack_access	fp_off within frame bounds	invalid indirect read
helper_arg_type	type==expected_arg_type	R1 scalar expected pkt
scalar_bounds	smin>0 && umax<=limit	unbounded min value
reference_release	all refs released before exit	Unreleased reference id=N
context_access	off+sz<=ctx_size	invalid bpf_context access
alignment	off % align == 0	misaligned access
type_safety	type matches expected	R1 inv expected fp

`reg.off + access_size <= reg.range`; BPFix infers this from the register fields at the rejection point.

Inference algorithm. For each error ID, BPFix defines: (1) which register(s) contain the failing object (extracted from the rejection line), (2) which predicate template applies (from the proof family), and (3) how to extract concrete parameters from the register state dump. The resulting predicate is a boolean function over register state that can be evaluated at any point in the trace.

3.4 Abstract State Transition Detection

The core of BPFix is detecting *safety-relevant abstract state transitions* across the verifier’s execution trace. Given the instantiated predicate P from Section 3.3 as a focus mechanism, BPFix evaluates P against the abstract state at every instruction, producing a status sequence:

$$\underbrace{?, \dots, ?}_{\text{unknown}}, \underbrace{\top, \dots, \top}_{\text{satisfied}}, \underbrace{\perp, \dots, \perp}_{\text{violated}}$$

Each transition in this sequence corresponds to a concrete abstract state change (e.g., a bounds collapse, type downgrade, or provenance loss) at a specific instruction. BPFix labels each transition:

- *proof_established*: the abstract state transition where P becomes satisfied ($? \rightarrow \text{true}$)—e.g., a bounds check narrows a register’s range
- *proof_holds*: instructions where the abstract state preserves P
- *proof_lost*: the abstract state transition where P goes from satisfied to violated ($\text{true} \rightarrow \text{false}$)—the *transition witness*, e.g., an OR instruction collapses bounds
- *rejected*: the final error instruction (always identified)

If P is never satisfied, the status is *never_established* (a source bug: the required abstract state was never produced, indicating a missing safety check). If P is satisfied but a subsequent state transition destroys it before the rejection, the status is *established_then_lost* (a lowering artifact: the source has a valid check but compiler-generated instructions

produce an abstract state transition that the verifier's domain cannot track).

Key distinction from heuristic approaches. The transition witness is not “detect a bounds collapse pattern.” It is “evaluate the specific safety predicate at each abstract state and find the exact instruction whose state transition broke it.” Different predicates identify different transition witnesses on the same trace. This is formally grounded in the verifier's abstract domain, not in ad hoc pattern detection.

Register value lineage. Predicates reference registers, but register names are reused across the trace. A value computed in R0 may be copied to R3 before the access. BPFix tracks value identity across register moves and copies. If R3 is defined as a copy of R0 ($r_3 = r_0$), and the proof predicate references the packet range of R0, BPFix continues tracking the range on R3 for subsequent instructions. This register value lineage is essential for correctly identifying proof-established and proof-lost events across LLVM's register allocation.

Proof propagation analysis. For lowering artifact detection, BPFix additionally checks whether the proof established at the bounds-check site *propagates* to the register used at the access site. If a different register is used for the access than the one whose range was established at the check (a common LLVM pattern), BPFix classifies this as a lowering artifact regardless of whether the proof predicate was technically satisfied at some earlier instruction.

3.5 Formal Foundation: Abstract State Transition Analysis

The state transition detection described above can be understood as a *second-order abstract interpretation*: BPFix applies abstract interpretation to the *output* of another abstract interpreter [1]. This section formalizes the framework, which applies to any abstract interpreter that emits per-step states.

Two layers of abstract interpretation. Layer 1 is the eBPF verifier itself. Its input is BPF bytecode; its abstract domain D_1 consists of register types, scalar bounds $[um_{in}, um_{ax}]$, pointer offsets, and packet ranges. Its output is a trace $T = (s_0, s_1, \dots, s_n)$ where each $s_i \in D_1$ is the abstract state after the i -th instruction.

Layer 2 is BPFix. Its input is the trace T ; it analyzes abstract state transitions (s_{i-1}, s_i) at each step, focused by a safety predicate P , and outputs a sequence of proof lifecycle labels that identify where and how the proof broke.

Definition 1 (Proof status lattice). *The proof status lattice is $\mathcal{L} = \{\perp, unknown, satisfied, violated\}$ with ordering $\perp \sqsubseteq unknown \sqsubseteq satisfied$ and $\perp \sqsubseteq unknown \sqsubseteq violated$, where *satisfied* and *violated* are incomparable. The bottom element $\perp$ represents an uninitialized status (relevant registers not yet present in the abstract state); *unknown* represents a state where the predicate is evaluable but the relevant registers are not*

yet constrained. Each transition between lattice elements corresponds to a safety-relevant abstract state transition in the verifier trace.

Definition 2 (State transition transfer function). *Let $P : D_1 \rightarrow \{true, false, unknown\}$ be the safety predicate instantiated from the rejection point (§3.3). For each traced instruction i with verifier state $s_i \in D_1$, the transfer function $\tau_i : \mathcal{L} \rightarrow \mathcal{L}$ maps the previous proof status to the status after observing the abstract state transition at instruction i :*

$$\tau_i(\ell) = \begin{cases} sat & P(s_i) \wedge \ell \in \{unk, sat\} \\ vio & \neg P(s_i) \wedge \ell = sat \\ \ell & otherwise \end{cases}$$

The status sequence is computed by forward iteration: $\ell_0 = unknown$, $\ell_i = \tau_i(\ell_{i-1})$.

Definition 3 (Transition witness). *The transition witness is the smallest index w such that $\ell_{w-1} = satisfied$ and $\tau_w(\ell_{w-1}) = violated$. This is the exact instruction where the proof breaks. If no such w exists and $\ell_n = violated$, the proof was never established.*

Theorem 1 (Required-proof lifecycle soundness). *Assume the terminal rejection is mapped to the correct required proof P , and the monitor for P is sound with respect to the verifier check it represents: when the monitor reports satisfied at abstract state s_i , every concrete state $c \in \gamma(s_i)$ satisfies the checked safety condition, and when it reports violated, the verifier lacks the abstract evidence needed to discharge that check at i . Then every lifecycle label emitted by BPFix corresponds to a real change in the verifier's abstract evidence for P : *proof_established* marks a transition from missing to established evidence, *proof_lost* marks a transition from established to missing or insufficient evidence, and *rejected* marks a final state that does not discharge P .*

Proof sketch. The verifier is a sound abstract interpreter: each s_i over-approximates all concrete states reachable at instruction i [4]. The required-proof monitor maps each s_i to the proof-status lattice by the per-family predicate for P . By the monitor-soundness assumption, each per-state status is a sound statement about the verifier's available evidence for the corresponding check. The lifecycle labels are emitted exactly at the status transitions computed by the transfer function τ_i . Therefore the labels identify real changes in verifier abstract evidence for P , though they do not by themselves prove semantic root cause or repair correctness. $\square$

Proof-relevant backward slice. Given the transition witness w , the *proof-relevant backward slice* is

$$S = \{i < w \mid \exists r \in \text{regs}(P). \text{instruction } i \text{ writes to } r\}$$

i.e., all instructions before w that write to registers appearing in the proof predicate P . When `mark_precise` backtracking information is available in the trace, S is refined to include only instructions on the backtracking chain, yielding a tighter slice.

Generalization beyond eBPF. Abstract state transition analysis applies to any abstract interpreter that outputs per-step abstract states. The requirements are: (1) per-step abstract states are emitted, (2) safety properties are expressible as predicates over the abstract domain, and (3) the trace is ordered. The key insight is domain-independent: given a sequence of abstract states $[s_0, \dots, s_n]$ and a safety predicate P , detecting the transition (s_{w-1}, s_w) where P changes from satisfied to violated locates the proof-loss point regardless of what the abstract domain represents. Rust’s borrow checker (lifetime predicates over borrow state), WebAssembly validators (type predicates over operand stack state), and Java bytecode verifiers (type predicates over stack frame state) all satisfy these requirements—suggesting that abstract state transition analysis is a general technique for improving diagnostic quality in verified language runtimes.

3.6 BTF Source Correlation

BPFix maps each detected state transition event (proof established, proof lost, rejected) to its source location via BTF `line_info` annotations [8]. When the verifier emits a source-location comment before an instruction (e.g., `; foo.c:42`), BPFix records the mapping. Multiple consecutive bytecode instructions from the same source line are merged into a single span.

When BTF annotations are absent, BPFix falls back to bytecode-level spans (instruction indices only). Object files are optional enrichments: when an object path is available, `bpfanalysis` can decode BPF instruction sections, build ProgramCFG summaries, and attach verifier states to CFG sites when the verifier PC layout matches the object section layout.

3.7 Diagnostic Rendering

BPFix produces two output formats from the same state transition analysis:

Human-readable Rust-style output. Modeled on Rust compiler diagnostics [12], the output shows multiple source spans with causal labels, register state changes at each transition, and note/help annotations. The format is designed to be readable in a terminal, an IDE, or a CI log without any tool support.

Structured JSON. A machine-readable format containing `error_id`, `taxonomy_class`, `proof_status`, an array of spans (each with role, source location, instruction index, and state change), the instantiated proof predicate, and note/help text. This format is designed for consumption by LLM agents (replacing raw log text in prompts), CI systems (structured failure annotations), and IDE plugins.

3.8 Implementation Notes

BPFix is implemented as a Rust Cargo workspace. The `bpfix` crate owns the command-line interface, required-proof reconstruction, user-facing failure classes, text rendering, and JSON rendering. The `bpfanalysis` crate owns neutral facts: verifier-log state parsing, BPF instruction modeling, bytecode lifting, ProgramCFG summaries, and verifier-state attachment.

Log-first operation. BPFix requires only verifier log text for its basic diagnostic path. Object files, source files, and external debug information are optional enrichments. This matches the intended workflow: developers keep their current loader or build system and pass the captured verifier log to `bpfix`.

Language independence. Because BPFix analyzes verifier logs and BPF bytecode-level facts, it is agnostic to whether the original program was written in C, Rust, Go, or another language. The methodology in Section 4 can stratify diagnostics by source language when benchmark metadata records it.

4 Evaluation Methodology

We evaluate BPFix on the versioned `bpfix-bench` corpus. The benchmark manifest is the case registry: each admitted case identifies a verifier log and, when available, source metadata or an object file. Runtime diagnosis consumes only the verifier log and optional object path. Benchmark labels are used after diagnosis as evaluation oracles, not as inputs to the tool.

The evaluation is organized around six questions:

- Q1** Does BPFix generate diagnostics reliably over admitted benchmark logs?
- Q2** Which verifier proof families can BPFix reconstruct, and which terminal rejections remain unknown?
- Q3** Do emitted lifecycle spans identify useful source or bytecode locations for the rejected verifier proof?
- Q4** How much does optional object-CFG correlation add beyond log-only diagnostics?
- Q5** How does BPFix compare with final-line and Pretty Verifier baselines on the same admitted corpus?
- Q6** What is the runtime overhead of log-only and object-enriched diagnosis?

4.1 Corpus

`bpfix-bench/manifest.yaml` is the canonical discovery entry point for the corpus. The corpus contains replayable verifier-failure cases from kernel selftests, Stack Overflow questions, GitHub issues, and GitHub commits. A case is admitted when its manifest entry points to a readable verifier log and the log contains a terminal verifier rejection. Optional fields may provide source files, object files, expected error annotations, fix locations, or language metadata; these fields are used only for stratified analysis and scoring.

4.2 Metrics

Reliability. We measure whether BPFix exits successfully and emits a structured diagnostic for each admitted case. Parser failures, object-analysis failures, and unknown terminal rejections are reported separately so that log-only diagnosis is not conflated with optional enrichment.

Required-proof reconstruction. We classify each diagnostic by the proof family in Table 1. The metric records whether a known family was reconstructed, whether the terminal rejection fell into an unknown family, and whether the rejection was outside verifier-proof diagnosis scope.

Localization quality. For cases with source, bytecode, or expected error annotations, we compare BPFix spans against the benchmark oracle. The primary spans are the rejected instruction, the proof-established site, and the proof-lost site when such lifecycle events exist in the trace.

Object correlation. For cases with object files, we run both log-only and object-enriched diagnosis. The object-enriched run measures whether BPF instructions are decoded, whether a ProgramCFG is built, and whether verifier states attach to CFG sites without changing the log-only diagnosis.

Baseline comparison. We compare BPFix with two baselines on the same admitted corpus: a final-line classifier that sees only the terminal verifier error, and Pretty Verifier [9]. Baselines are scored with the same reliability and localization criteria where their outputs provide comparable fields.

Runtime overhead. We measure wall-clock latency for log-only and object-enriched modes on the same host. The protocol records input size, object size when present, and whether the run used warm filesystem cache.

4.3 Protocol

For each admitted case, the evaluation first runs BPFix in log-only mode and records exit status, diagnostic ID, proof family, lifecycle events, spans, evidence notes, and runtime. If the manifest provides an object file, the evaluation then runs the object-enriched path and records object decoding, CFG construction, and state-attachment outcomes. The YAML label is consulted only after both runs complete.

All baselines are executed on the same admitted cases. Their outputs are normalized into the shared scoring schema where possible: terminal failure class, source or bytecode location, and whether they produce an actionable diagnostic rather than an unmanaged-error fallback. Quantitative results are reported only from this protocol.

5 Discussion and Limitations

5.1 Scope and Honest Limitations

Proof-family coverage. Table 1 enumerates the verifier proof families reconstructed by BPFix. Unknown terminal rejections receive fallback diagnostics, while family-specific

lifecycle analysis requires a monitor for the corresponding verifier proof.

Source correlation depends on input richness. Log-only mode can emit source spans only when the verifier log already contains source comments. When the log lacks source context, BPFix falls back to bytecode PCs and log-line evidence. Object-CFG correlation is implemented as optional metadata; full BTF-backed source recovery from object files is outside the methodology in this paper.

Path-insensitive slicing. BPFix’s backward slice from the transition witness is bounded and path-insensitive. It will not correctly handle all loop- or alias-sensitive proof loss patterns. We scope the slice as an explanatory aid after the main predicate/transition detection, and validate it empirically rather than claiming formal precision.

Repair synthesis is out of scope. BPFix diagnoses verifier failures; it does not synthesize patches or run a verifier-oracle repair loop. Repair evaluation belongs to a separate capability with its own verifier-pass oracle.

Kernel-version dependence. The verifier’s log format varies across kernel versions. Prior work [15] finds that 83% of eBPF extensions are affected by at least one kernel mismatch; deployment studies should therefore record kernel version as part of the evaluation protocol.

5.2 Upstream Potential

BPFix’s output could become part of the kernel’s own diagnostic infrastructure. The structured JSON format and stable error IDs are designed to be extensible. The required-proof catalog could inform improvements to verifier error messages. The Rust-style rendering could be integrated into bpf tool or the libbpf loader, so developers receive structured diagnostics directly from the load path.

6 Related Work

6.1 eBPF Verifier Complexity and Developer Pain

Jia et al. [7] argue that kernel-extension verification is “untenable” as the extension surface grows; BPFix addresses the near-term reality that millions of C/Rust eBPF programs run against the current verifier. Rex [6] studies 72 verifier workaround commits, confirming 44.4% are LLVM code-generation changes; Rex explains *why* such failures exist, BPFix explains *this specific failure instance* at load time. Dekar et al. [2] find 19.3% of 743 Stack Overflow questions are verifier-related, quantifying the pain point BPFix addresses.

6.2 Diagnostic Tools

Pretty Verifier [9] applies regex patterns to the final verifier error line, producing a single-span explanation; it does not reconstruct the lifecycle of the verifier proof that led to the final rejection. BPFix analyzes verifier state evidence across the trace—a different level of analysis that supports required-proof diagnostics. The ebpf-verifier-errors

repository [11] crowdsources (log, fix) pairs as data, but provides no automated analysis.

6.3 LLM-Based eBPF Tools

Kgent [16] feeds raw verifier output verbatim to an LLM repair loop; BPFix's structured diagnostics could serve as Kgent's preprocessor, replacing unstructured register dumps with labeled causal spans. SimpleBPF [3] generates a high-level DSL and explicitly notes that raw-log retry "was not effective"; BPFix addresses the general case (arbitrary handwritten C/Rust) that DSLs cannot replace. K2 [14] optimizes eBPF programs for performance, a goal orthogonal to failure diagnosis.

6.4 Static Analysis and Diagnostics

BPFix's approach is inspired by counterexample analysis in model checking: reconstruct why verification failed from the evidence trace. Abstract interpretation [1] and its formalization for eBPF [4] ground BPFix's state transition analysis. Program slicing [13] motivates backward dependency extraction from the transition witness. Rust's compiler diagnostics [12] pioneered multi-span source-level errors with causal labels; research on compiler error messages [5] confirms that pointing at the wrong location significantly harms developer productivity.

7 Conclusion

The eBPF verifier already emits much of the evidence needed to explain why a program is rejected, but that evidence is buried in verbose traces. BPFix turns that evidence into Rust-style diagnostics by reconstructing the required verifier proof and tracking its lifecycle across verifier abstract states. The tool works in userspace with existing eBPF development workflows and requires no kernel modifications.

References

- [1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *SIGPLAN Notices* (1977), 238–252. <https://doi.org/10.1145/512950.512973>
- [2] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 1–8. <https://doi.org/10.1145/3672197.3673429>
- [3] Xiangyu Gao, Xiangfeng Zhu, Bhavana Vannarth Shobhana, Yiwei Yang, Arvind Krishnamurthy, and Ratul Mahajan. 2025. Offloading the Tedious Task of Writing eBPF Programs. In *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions*. 63–69. <https://doi.org/10.1145/3748355.3748369>
- [4] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 1–15. <https://doi.org/10.1145/3314221.3314590>
- [5] Brett Hartmann, Mark Deline, and Peter Lee. 2014. Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. In *Proceedings of the Workshop in Primary and Secondary Computing Education (WiPSCE)*. 75–84. <https://doi.org/10.1145/2670757.2670780>
- [6] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 325–342. <https://www.usenix.org/conference/atc25/presentation/jia>
- [7] Jinghao Jia, Raj Sahu, Adam Oswald, Dan Williams, Michael V. Le, and Tianyin Xu. 2023. Kernel Extension Verification is Untenable. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*. 150–157. <https://doi.org/10.1145/3593856.3595892>
- [8] Linux Kernel Documentation. 2024. BPF Type Format (BTF). Linux kernel documentation. <https://docs.kernel.org/bpf/btf.html>
- [9] Sebastiano Miano, Roberto Lezzi, Gabriele Lospinoso, and Fulvio Rizzo. 2025. Pretty Verifier: Towards Friendly eBPF Verification Errors. GitHub repository. <https://github.com/netgroup-polito/pretty-verifier> Unpublished, accessed 2026-03-12.
- [10] NCC Group. 2024. *eBPF Verifier Code Audit*. Technical Report. The Linux Foundation. <https://www.linuxfoundation.org/press/linux-foundation-releases-open-source-technology-security-audit-of-the-ebpf-verifier>
- [11] parttimenerd. 2024. *ebpf-verifier-errors*. GitHub repository. <https://github.com/parttimenerd/ebpf-verifier-errors> Accessed 2026-03-12.
- [12] Rust Language Team. 2016. Improved Error Messages in the Rust Compiler. The Rust Blog. <https://blog.rust-lang.org/2016/08/10/Shape-of-errors-to-come.html> Accessed 2026-03-12.
- [13] Mark Weiser. 1981. Program Slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*. 439–449.
- [14] Qiongwen Xu, Michael D. Brown, Tao Li, Chengbin Pang, Bhavna Sopori, Sahil Jain, Junfeng Yang, and Arpit Gupta. 2023. K2: Automated eBPF Program Optimization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 1–15. <https://doi.org/10.1145/3603269.3604882>
- [15] Junxiao Zhang, Samantha Mathews, Hiep Nguyen, Ryan Stutsman, and Asaf Cidon. 2025. DepSurf: Measuring and Analyzing the Dependency Surface of Kernel Extensions. Project site and paper. <https://sites.google.com/view/depsurf-bpf/home>
- [16] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 30–36. <https://doi.org/10.1145/3672197.3673434>